

3 Quy tắc vàng để ghép code vô trùng 100%

Để hạn chế tối đa lỗi khi nhiều thành viên cùng phát triển một dự án Django, nhóm thống nhất áp dụng **3 quy tắc vàng** sau đây.

Quy tắc 1: Cấm đặt tên định danh chung chung

Strict Naming

Trong cùng một ứng dụng, tuyệt đối không sử dụng các tên class chung chung như:

```
TaskSerializer
TaskViewSet
```

Thay vào đó, bắt buộc phải gắn tiền tố theo vai trò của từng nhóm chức năng.

Quy ước đặt tên

Vai trò	Serializer	ViewSet
Manager	ManagerTaskSerializer	ManagerTaskViewSet
Employee	EmployeeTaskSerializer	EmployeeTaskViewSet
Admin	AdminTaskSerializer	AdminTaskViewSet

Lý do

Việc đặt tên rõ ràng theo vai trò giúp tránh tình trạng trùng tên class trong Python.

Ví dụ, nếu nhiều người vô tình viết chung vào một file, Python sẽ không bị lỗi hoặc ghi đè class do trùng tên như:

```
class TaskSerializer:
    pass
```

Thay vào đó, mỗi class có tên riêng biệt:

```
class ManagerTaskSerializer:
    pass

class EmployeeTaskSerializer:
    pass
```

```
class AdminTaskSerializer:
    pass
```

Quy tắc 2: Tách file vật lý theo vai trò

Không để cả 3 thành viên cùng chỉnh sửa trực tiếp các file mặc định như:

```
tasks/views.py
tasks/serializers.py
tasks/urls.py
```

Thay vào đó, bên trong từng app, mỗi vai trò phải có file riêng.

Cấu trúc file đề xuất

```
tasks/
|
|— views_manager.py
|— serializers_manager.py
|— urls_manager.py
|
|— views_employee.py
|— serializers_employee.py
|— urls_employee.py
|
|— views_admin.py
|— serializers_admin.py
|— urls_admin.py
|
|— views.py
|— serializers.py
|— urls.py
```

Quy định

Các file mặc định của Django như:

```
views.py
serializers.py
urls.py
```

sẽ để trống hoặc không sử dụng trực tiếp. Không thành viên nào được chỉnh sửa các file này nếu không có thống nhất chung.

Lý do

Mỗi thành viên làm việc trong file riêng của mình:

Vai trò	File view	File serializer	File url
Manager	<code>views_manager.py</code>	<code>serializers_manager.py</code>	<code>urls_manager.py</code>
Employee	<code>views_employee.py</code>	<code>serializers_employee.py</code>	<code>urls_employee.py</code>
Admin	<code>views_admin.py</code>	<code>serializers_admin.py</code>	<code>urls_admin.py</code>

Khi đẩy code lên GitHub, các file được tách riêng nên Git có thể tự động gộp code dễ hơn. Điều này giúp giảm tối đa lỗi conflict khi merge code.

Quy tắc 3: Quy hoạch tại trạm tổng

`worktracker_core/urls.py`

File `worktracker_core/urls.py` là file định tuyến trung tâm của toàn bộ hệ thống.

Đây là nơi duy nhất cả 3 thành viên có khả năng phải chỉnh sửa, vì vậy cần chia khu vực rõ ràng theo từng vai trò.

Ví dụ cấu trúc `worktracker_core/urls.py`

```
"""
worktracker_core/urls.py - Trạm định tuyến trung tâm
"""

from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),

    # ===== KHU VỰC CỦA ADMIN (MINH ANH) =====
    # path('api/v1/admin/users/', include('accounts.urls_admin')),

    # ===== KHU VỰC CỦA MANAGER (LONG) =====
    path('api/v1/manager/', include('tasks.urls_manager')),
    path('api/v1/manager/', include('timesheets.urls_manager')),

    # ===== KHU VỰC CỦA EMPLOYEE (TÚ) =====
    # path('api/v1/employee/tasks/', include('tasks.urls_employee')),
]
```

Quy định khi chỉnh sửa

Mỗi thành viên chỉ được thêm hoặc sửa route trong khu vực của mình.

Ví dụ:

- Admin chỉ chỉnh trong khu vực `ADMIN`
- Manager chỉ chỉnh trong khu vực `MANAGER`
- Employee chỉ chỉnh trong khu vực `EMPLOYEE`

Không tự ý chỉnh sửa route thuộc khu vực của thành viên khác nếu chưa thống nhất.

Kết luận

Để ghép code an toàn và hạn chế conflict, nhóm cần tuân thủ 3 nguyên tắc chính:

1. **Đặt tên class rõ ràng theo vai trò**
2. **Tách file vật lý theo vai trò**
3. **Chia khu vực rõ ràng trong file** `worktracker_core/urls.py`

Nếu tuân thủ đúng 3 quy tắc này, việc làm việc nhóm trên GitHub sẽ ổn định hơn, dễ merge hơn và giảm đáng kể lỗi trùng code hoặc conflict khi phát triển dự án.